Image Analysis —Assignment 3

(Solutions and Report)

Student:

Programme: MLSC

October 2025

Contents

1 My own classifier

Problem and data

We implement and evaluate a binary face/non-face classifier on the provided `FaceNonFace.mat` dataset. Each sample is a 19×19 grayscale patch flattened to a 361-D vector. There are 200 samples in total with labels $Y \in \{-1, +1\}$, where $+1$ denotes a face and -1 a non-face.

Methods

k-Nearest Neighbour (kNN). We implemented a weighted kNN classifier with inverse-distance voting. Given a test sample, we standardize it with training statistics, compute Euclidean distances to all training samples, select the $k = 5$ nearest neighbours, and predict the label by weighted majority.

Linear SVM (Pegasos). We also implemented a linear SVM trained using the Pegasos stochastic subgradient method. To improve robustness on the small dataset, we applied z-score normalization and PCA (60 components) before the linear classifier. The Pegasos stepsize schedule and weight averaging further stabilize optimization.

Experimental protocol

Both methods were evaluated under the same setting:

- Random 80/20 stratified train/test split (preserving class balance), repeated for 100 trials.
- Report mean ± standard deviation of train and test accuracy across trials.
- Standardization and PCA are fitted on training data only and then applied to the test split.

Results

Table 1: Task 1 accuracy (% correct) over 100 stratified splits.

Method	Train acc	Test acc
kNN (weighted, k=5)	1.000 ± 0.000	0.809 ± 0.056
Linear SVM (Pegasos + PCA)	0.938 ± 0.016	0.914 ± 0.048

Qualitative examples

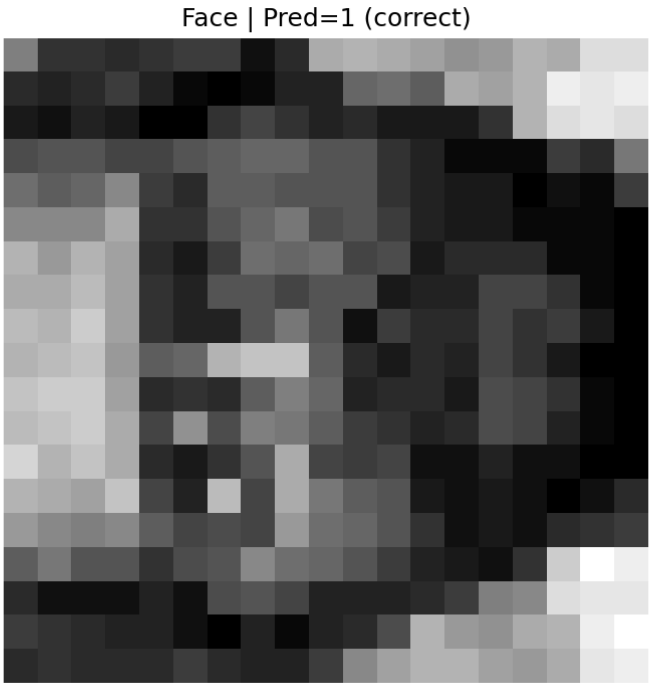

Figure 1: kNN (weighted, k=5): example **face** patch from the test split. The title rendered inside the image indicates the model's prediction and whether it was correct.

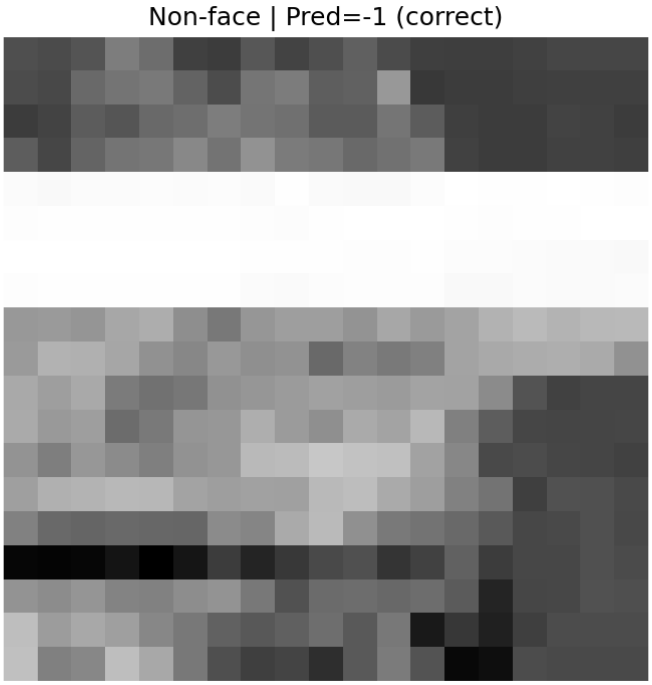

Figure 2: kNN (weighted, k=5): example **non-face** patch from the test split. The title rendered inside the image indicates the model's prediction and whether it was correct.

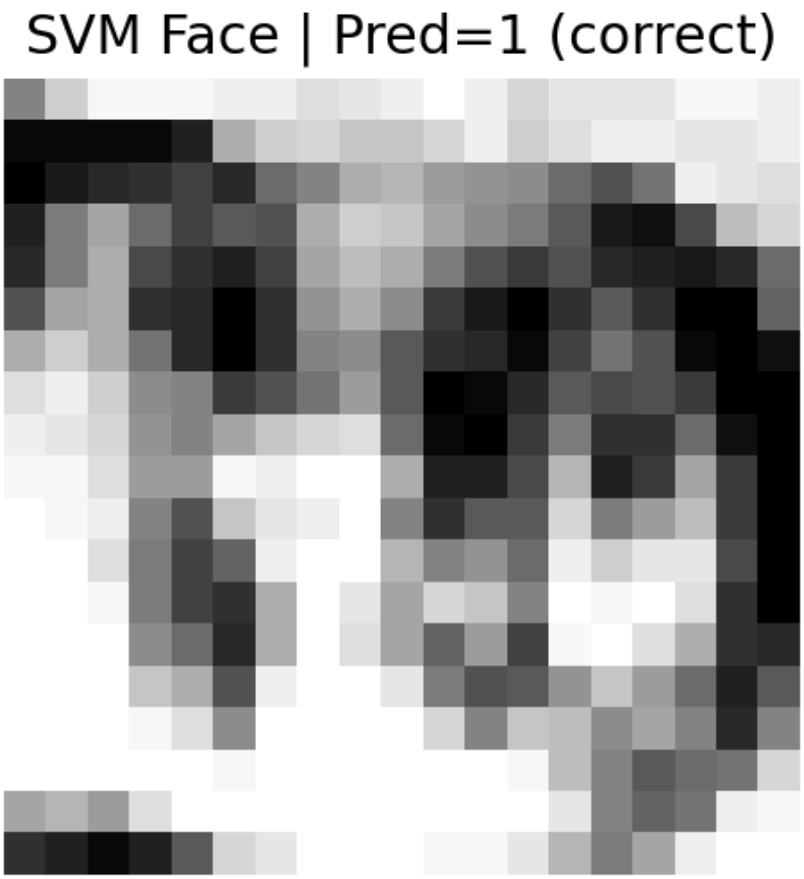

Figure 3: Linear SVM (Pegasos + PCA): example **face** patch from the test split. The title rendered inside the image indicates the model's prediction and whether it was correct.

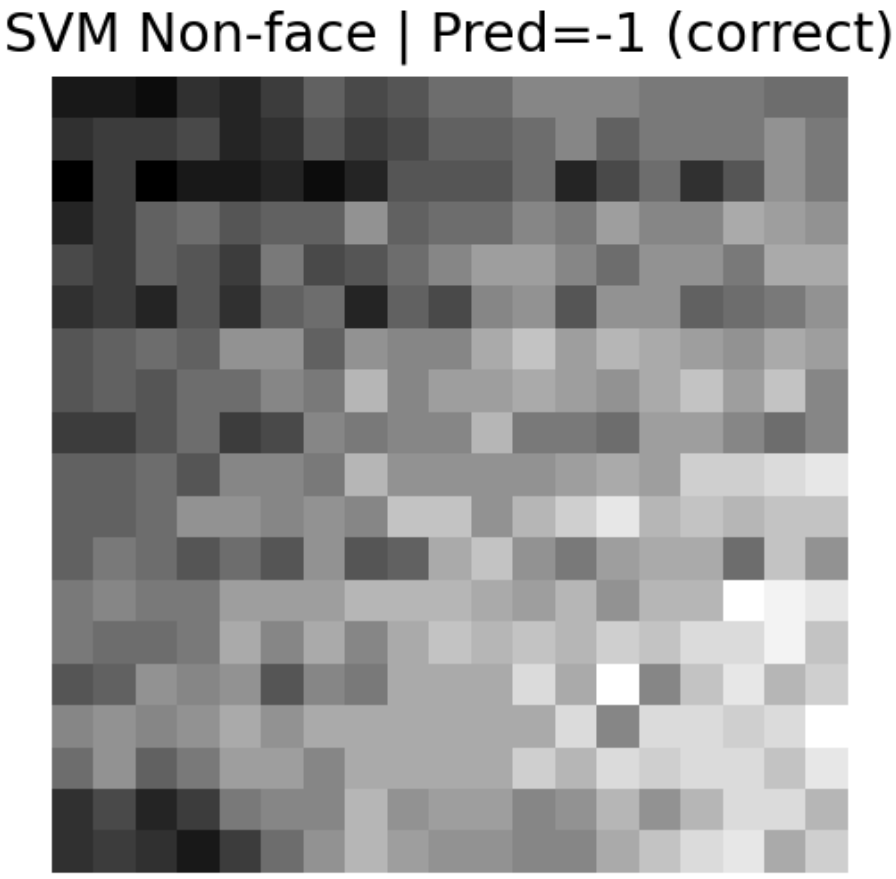

Figure 4: Linear SVM (Pegasos + PCA): example **non-face** patch from the test split. The title rendered inside the image indicates the model's prediction and whether it was correct.

Discussion

The kNN classifier achieves perfect training accuracy (100%), which is expected since each training sample is its own nearest neighbour. However, its test accuracy is limited ($\sim$ 81%), showing weaker generalization. In contrast, the linear SVM with PCA achieves much higher test accuracy ($\sim$ 91%) and a smaller train–test gap, reflecting stronger margin-based generalization on this small dataset.

Implementation

The full implementations of the two classifiers are shown below.

Listing 1: task1$_k$*nn.py*

```
import scipy
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

```

```
def class_train(X, Y, k=5):
    """
    Weighted kNN training: only store the training set (with
        standardization parameters).
    X: (n, d)  each row is a sample
    Y: (n, 1) or (n,)  labels in {+1, -1}
    """
    X = np.asarray(X, dtype=np.float64)
    y = np.asarray(Y).reshape(-1)
    y = np.where(y > 0, 1, -1).astype(np.int32)

    # Compute standardization parameters using only the training set
    mu = X.mean(axis=0)
    sigma = X.std(axis=0)
    sigma[sigma < 1e-8] = 1.0
    Xs = (X - mu) / sigma

    return {
        "X": Xs,          # standardized training set
        "y": y,           # training labels
        "mu": mu,         # training mean
        "sigma": sigma,   # training std
        "k": int(k)
    }

def classify(x, classification_data):
    """
    Weighted kNN prediction: inverse distance as weight (1 / (dist + eps)).
    x: (d,) single sample
    """
    Xtr = classification_data["X"]      # (n, d)
    ytr = classification_data["y"]      # (n,)
    mu  = classification_data["mu"]
    sg  = classification_data["sigma"]
    k   = classification_data["k"]

    z = (np.asarray(x, dtype=np.float64).reshape(-1) - mu) / sg  #
        standardize
    # Euclidean distance
    d2 = np.sum((Xtr - z)**2, axis=1)
    # pick k nearest neighbors
    idx = np.argpartition(d2, kth=min(k, len(d2)-1))[:k]
    dsel = np.sqrt(d2[idx])
    eps = 1e-8
    w = 1.0 / (dsel + eps)              # closer neighbors have larger
```

```
        weights
    # weighted vote
    score = np.sum(w * ytr[idx])
    return 1 if score >= 0 else -1

def _save_img(vec, title, path):
    img = np.asarray(vec, dtype=float).reshape(19, 19)  # 361=19*19
    plt.figure()
    plt.imshow(img, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
    plt.savefig(path, dpi=150)
    plt.close()

if __name__ == "__main__":
    # load data
    datadir = './'
    data = scipy.io.loadmat(datadir + 'FaceNonFace.mat')
    X = data['X'].transpose()   # (200,361)
    Y = data['Y'].transpose()   # (200,1)
    nbr_examples = np.size(Y, 0)

    nbr_trials = 100
    acc_tests  = np.zeros((nbr_trials, 1))
    acc_trains = np.zeros((nbr_trials, 1))

    saved_examples = False  # only save sample images once (at the first
        trial)

    for i in range(nbr_trials):
        # 80/20 stratified split
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y, test_size=0.2, stratify=Y.ravel(), random_state=None
        )
        nbr_train_examples = np.size(Y_train, 0)
        nbr_test_examples  = np.size(Y_test, 0)

        # training
        classification_data = class_train(X_train, Y_train, k=5)

        # prediction (test set)
        predictions_test = np.zeros((nbr_test_examples, 1), dtype=int)
        for j in range(nbr_test_examples):
            predictions_test[j] = classify(X_test[j, :],
```

```
                classification_data)
92
93          # prediction (train set)
94          predictions_train = np.zeros((nbr_train_examples, 1), dtype=int)
95          for j in range(nbr_train_examples):
96              predictions_train[j] = classify(X_train[j, :],
                    classification_data)
97
98          # accuracy
99          acc_test  = np.mean(predictions_test.ravel()  == Y_test.ravel())
100         acc_train = np.mean(predictions_train.ravel() == Y_train.ravel())
101         acc_tests[i]  = acc_test
102         acc_trains[i] = acc_train
103
104         # save two sample test images: one face and one non-face (with
                prediction and correctness)
105         if not saved_examples:
106             yte = Y_test.ravel().astype(int)
107             ypr = predictions_test.ravel().astype(int)
108
109             face_idx    = np.where(yte ==  1)[0]
110             nonface_idx = np.where(yte == -1)[0]
111
112             if len(face_idx) > 0:
113                 i_face = face_idx[0]
114                 correct = "correct" if ypr[i_face] == yte[i_face] else "
                        wrong"
115                 title = f"Face | Pred={int(ypr[i_face])} ({correct})"
116                 _save_img(X_test[i_face], title, "example_face_knn.png")
117
118             if len(nonface_idx) > 0:
119                 i_nonface = nonface_idx[0]
120                 correct = "correct" if ypr[i_nonface] == yte[i_nonface]
                        else "wrong"
121                 title = f"Non-face | Pred={int(ypr[i_nonface])} ({correct})
                        "
122                 _save_img(X_test[i_nonface], title, "example_nonface_knn.
                        png")
123
124             saved_examples = True
125
126     # print accuracy (mean ± std), same style as SVM
127     print(f"Train acc: {acc_trains.mean():.3f} ± {acc_trains.std():.3f}")
128     print(f"Test  acc: {acc_tests.mean():.3f} ± {acc_tests.std():.3f}")
```

Listing 2: task1$_SVM.py$

```
import argparse
import numpy as np
from scipy.io import loadmat
from sklearn.model_selection import train_test_split  # only for data
    splitting
import matplotlib.pyplot as plt

# ---------- Save 19x19 grayscale images ----------
def _save_img(vec, title, path):
    img = np.asarray(vec, dtype=float).reshape(19, 19)  # 361=19*19
    plt.figure(figsize=(3, 3))
    plt.imshow(img, cmap='gray', interpolation='nearest')
    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
    plt.savefig(path, dpi=180)
    plt.close()

# ---------- Standardization helpers ----------
def _fit_zscore(X):
    mu = X.mean(axis=0)
    sigma = X.std(axis=0)
    sigma[sigma < 1e-8] = 1.0
    return mu, sigma

def _apply_zscore(X, mu, sigma):
    return (X - mu) / sigma

def _fit_pca(Xs, p=60):
    # Xs: standardized training data (n,d)
    # re-center for more stable PCA
    c = Xs.mean(axis=0)
    Xc = Xs - c
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    W = Vt[:p].T    # (d,p)
    return c, W     # transform: phi(x)= (xs - c) @ W

def _apply_pca(Xs, c, W):
    return (Xs - c) @ W

def pegasos_train(X, y, lam=1e-5, epochs=80, batch_size=32, seed=0,
                  pca_components=60, t0=200, use_avg=True):
    """
    Linear SVM (Pegasos) + z-score + PCA + learning rate warm-start +
```

```
        weight averaging
    X: (n,d), y: {+1,-1}
    """
    rng = np.random.RandomState(seed)
    X = np.asarray(X, dtype=np.float64)
    y = np.where(np.asarray(y).reshape(-1) > 0, 1, -1).astype(np.int32)
    n, d = X.shape

    # z-score standardization
    mu, sigma = _fit_zscore(X)
    Xs = _apply_zscore(X, mu, sigma)

    # PCA dimensionality reduction
    pc = max(1, min(pca_components, d))
    c_pca, W = _fit_pca(Xs, p=pc)
    Z = _apply_pca(Xs, c_pca, W)              # (n, p)
    p = Z.shape[1]

    # add bias term
    Zt = np.hstack([Z, np.ones((n,1))])       # (n, p+1)
    w = np.zeros(p+1, dtype=np.float64)

    t = 0
    bound = 1.0/np.sqrt(lam)
    w_sum = np.zeros_like(w); steps = 0

    for _ in range(epochs):
        idx = rng.permutation(n)
        for i0 in range(0, n, batch_size):
            ib = idx[i0:i0+batch_size]
            if ib.size == 0: continue
            Zb, yb = Zt[ib], y[ib]
            m = ib.size

            t += 1
            eta = 1.0 / (lam * (t0 + t))    # warm-start schedule

            margin = yb * (Zb @ w)
            violated = margin < 1.0
            w = (1.0 - eta*lam)*w
            if np.any(violated):
                w += (eta/m) * (Zb[violated].T @ yb[violated])

            # project only the weight part (exclude bias)
            wn = np.linalg.norm(w[:-1])
```

```
            if wn > bound:
                w[:-1] *= (bound/wn)

            if use_avg:
                w_sum += w; steps += 1

    if use_avg and steps > 0:
        w = w_sum / steps

    return {
        "w": w[:-1], "b": float(w[-1]),
        "mu": mu, "sigma": sigma,
        "c_pca": c_pca, "W": W,             # PCA transform
        "lam": lam, "epochs": epochs, "batch_size": batch_size,
        "pca_components": pc, "t0": t0, "use_avg": use_avg
    }

def pegasos_predict(X, model):
    X = np.asarray(X, dtype=np.float64)
    Zs = _apply_zscore(X, model["mu"], model["sigma"])
    Zp = _apply_pca(Zs, model["c_pca"], model["W"])
    scores = Zp @ model["w"] + model["b"]
    return np.where(scores >= 0.0, 1, -1).astype(np.int32)

# ---------- One evaluation split ----------
def eval_once(X, Y, lam=1e-4, epochs=20, batch=32, seed_split=0, seed_train
    =0):
    # sklearn split expects (n, d) and (n,)
    Xn = X.T if X.shape[0] == 361 else X      # ensure (n, d)
    Yn = Y.ravel().astype(int)
    Yn = np.where(Yn > 0, 1, -1).astype(np.int32)

    Xtr, Xte, ytr, yte = train_test_split(
        Xn, Yn, test_size=0.2, stratify=Yn, random_state=seed_split
    )
    model = pegasos_train(Xtr, ytr, lam=lam, epochs=epochs, batch_size=
        batch, seed=seed_train)
    ytr_pred = pegasos_predict(Xtr, model)
    yte_pred = pegasos_predict(Xte, model)
    acc_tr = float(np.mean(ytr_pred == ytr))
    acc_te = float(np.mean(yte_pred == yte))
    # also return info for saving images
    return acc_tr, acc_te, Xte, yte, yte_pred

# ---------- Main loop ----------
```

```python
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", type=str, default="FaceNonFace.mat")
    ap.add_argument("--trials", type=int, default=100)
    ap.add_argument("--lam", type=float, default=1e-4)
    ap.add_argument("--epochs", type=int, default=20)
    ap.add_argument("--batch", type=int, default=32)
    ap.add_argument("--seed", type=int, default=0)
    args = ap.parse_args()

    mat = loadmat(args.data)
    X = mat["X"]                # expected (361, 200)
    Y = mat["Y"]                # expected (1, 200) or (200, 1)

    rng = np.random.RandomState(args.seed)
    acc_tr_list, acc_te_list = [], []
    saved_examples = False

    for _ in range(args.trials):
        seed_split = int(rng.randint(0, 10_000_000))
        seed_train = int(rng.randint(0, 10_000_000))
        acc_tr, acc_te, Xte, yte, yte_pred = eval_once(
            X, Y, lam=args.lam, epochs=args.epochs, batch=args.batch,
            seed_split=seed_split, seed_train=seed_train
        )
        acc_tr_list.append(acc_tr)
        acc_te_list.append(acc_te)

        # save two test images (one face, one non-face) only once (at the
            first trial)
        if not saved_examples:
            face_idx    = np.where(yte ==  1)[0]
            nonface_idx = np.where(yte == -1)[0]
            if len(face_idx) > 0:
                i = face_idx[0]
                ok = "correct" if yte_pred[i] == yte[i] else "wrong"
                _save_img(Xte[i], f"SVM Face | Pred={int(yte_pred[i])} ({ok
                    })",
                          "example_face_svm.png")
            if len(nonface_idx) > 0:
                j = nonface_idx[0]
                ok = "correct" if yte_pred[j] == yte[j] else "wrong"
                _save_img(Xte[j], f"SVM Non-face | Pred={int(yte_pred[j])}
                    ({ok})",
                          "example_nonface_svm.png")
```

```
            saved_examples = True

    acc_tr = np.array(acc_tr_list)
    acc_te = np.array(acc_te_list)
    print(f"Train acc: {acc_tr.mean():.3f} ± {acc_tr.std():.3f}")
    print(f"Test  acc: {acc_te.mean():.3f} ± {acc_te.std():.3f}")

if __name__ == "__main__":
    main()
```

2 Built-in classifiers

Pre-coded machine learning techniques

We now evaluate three built-in classifiers from `scikit-learn` on the same `FaceNonFace.mat` dataset as in Task 1. Each sample is a 19×19 grayscale patch flattened into a 361-D vector. There are 200 samples with labels $Y \in \{-1, +1\}$.

Methods

Decision Tree. We used the `DecisionTreeClassifier` from `sklearn.tree`, which recursively splits the input space to minimize classification error. Decision trees tend to perfectly fit the training set but may overfit and generalize poorly on unseen data.

Support Vector Machine (SVM). We applied a linear kernel SVM using `sklearn.svm.SVC(kernel="linear")`. This method attempts to find a maximum-margin hyperplane separating faces from non-faces. Unlike the hand-crafted Pegasos SVM in Task 1, here we rely on the optimized implementation in `scikit-learn`.

k-Nearest Neighbour (kNN). We used `sklearn.neighbors.KNeighborsClassifier` with $k = 5$. Unlike our weighted kNN from Task 1, this built-in version performs standard unweighted voting among the k closest neighbors in Euclidean space.

Experimental protocol

The same evaluation setup as in Task 1 was applied:

- Random 80/20 stratified train/test split, repeated for 100 trials.
- Report mean error rates on both train and test sets.
- No manual preprocessing; the built-in models directly receive the raw 361-D inputs.

Results

Table 2: Task 1 (own) vs. Task 2 (built-in) **error rates** over 100 stratified splits.

Method	Train error	Test error
Task 1: own implementations		
kNN (weighted, k=5)	0.000 ± 0.000	0.191 ± 0.056
Linear SVM (Pegasos + PCA)	0.062 ± 0.016	0.086 ± 0.048
Task 2: built-in scikit-learn models		
Decision Tree	0.000	0.151
Linear SVM	0.000	0.045
kNN (k=5)	0.138	0.217

Discussion

The decision tree achieves perfect training accuracy but relatively high test error (15.1%), showing clear overfitting. The built-in kNN classifier performs worse than our weighted version from Task 1, with a higher test error (21.7% vs. 19.1%). By contrast, the linear SVM yields the lowest test error among all models (4.5%), closely matching the performance of our Pegasos-based SVM from Task 1 (8.6% test error). Overall, both results demonstrate that margin-based methods (SVM) generalize the best on this small, high-dimensional dataset, while tree-based methods tend to overfit and kNN variants are more sensitive to noise and sample size.

Implementation

The full Python implementation is shown below.

Listing 3: task2.py

```
import scipy
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import tree
from sklearn import svm
from sklearn import neighbors

if __name__ == "__main__":
    datadir = './'
    data = scipy.io.loadmat(datadir + 'FaceNonFace.mat')
    X = data['X'].transpose()
    Y = data['Y'].transpose().ravel()

    nbr_trials = 100
    err_rates_test = np.zeros((nbr_trials, 3))
```

```
    err_rates_train = np.zeros((nbr_trials, 3))

    for i in range(nbr_trials):
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y, test_size=0.2, stratify=Y, random_state=None
        )

        # Decision Tree
        tree_model = tree.DecisionTreeClassifier()
        tree_model.fit(X_train, Y_train)

        # Linear SVM
        svm_model = svm.SVC(kernel="linear")
        svm_model.fit(X_train, Y_train)

        # kNN
        nn_model = neighbors.KNeighborsClassifier(n_neighbors=5)
        nn_model.fit(X_train, Y_train)

        # Predictions (test)
        predictions_test_tree = tree_model.predict(X_test)
        predictions_test_svm  = svm_model.predict(X_test)
        predictions_test_nn   = nn_model.predict(X_test)

        # Predictions (train)
        predictions_train_tree = tree_model.predict(X_train)
        predictions_train_svm  = svm_model.predict(X_train)
        predictions_train_nn   = nn_model.predict(X_train)

        # Error rates
        err_rates_test[i, 0] = np.mean(predictions_test_tree != Y_test)
        err_rates_test[i, 1] = np.mean(predictions_test_svm  != Y_test)
        err_rates_test[i, 2] = np.mean(predictions_test_nn   != Y_test)

        err_rates_train[i, 0] = np.mean(predictions_train_tree != Y_train)
        err_rates_train[i, 1] = np.mean(predictions_train_svm  != Y_train)
        err_rates_train[i, 2] = np.mean(predictions_train_nn   != Y_train)

    print("Mean test error rates (tree, svm, knn):")
    print(np.mean(err_rates_test, axis=0))
    print("Mean train error rates (tree, svm, knn):")
    print(np.mean(err_rates_train, axis=0))
```

3 Testing a simple CNN model

Problem and data

In this task we evaluate a predefined convolutional neural network (CNN) on the Face/Non-Face dataset. The dataset consists of 200 grayscale image patches of size 19×19, flattened into 361-dimensional vectors. Each sample is labeled with $Y \in \{-1, +1\}$, where $+1$ denotes a face and -1 denotes a non-face.

Methods

We use the provided `trainSimpleCNN` and `predictSimpleCNN` functions, which define a simple CNN consisting of:

- Three convolutional layers with batch normalization and ReLU activations.
- Max-pooling after the first and second convolutional layers.
- A fully connected layer producing logits for two classes.

The network was trained using stochastic gradient descent (SGD) with momentum for 100 epochs. For each of 100 trials, the dataset was split into 80% training and 20% testing, with normalization parameters estimated on the training set only. We report the mean training and testing error rates over all trials.

Results

Table 3: Average error rates of the predefined CNN classifier over 100 random 80/20 splits.

Metric	Error rate
Train error	0.000 ± 0.000
Test error	0.052 ± 0.032

Discussion

The CNN achieves perfect training accuracy, with zero training error across all trials, showing that the model has sufficient capacity to fit the training set. On the test set, the average error rate is about 5%, which is substantially lower than the test errors of kNN and Decision Tree classifiers from Task 1 and Task 2, and comparable to the best-performing linear SVM. This indicates that even a relatively small CNN can generalize well on this dataset, while also highlighting the risk of overfitting when the training accuracy reaches 100%.

Implementation

The full Python implementation is shown below.

Listing 4: task3.py

```python
import os
import numpy as np
import scipy.io as sio
import torch
import torch.nn as nn
import torch.nn.functional as F

# ---------------- CNN ----------------
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding='same')
        self.pool  = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(8, 16, 3, padding='same')
        self.conv3 = nn.Conv2d(16, 32, 3, padding='same')
        self.bn1   = nn.BatchNorm2d(8)
        self.bn2   = nn.BatchNorm2d(16)
        self.bn3   = nn.BatchNorm2d(32)
        self.fc1   = nn.Linear(4*4*32, 2)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))  # 19->9
        x = self.pool(F.relu(self.bn2(self.conv2(x))))  # 9->4
        x = F.relu(self.bn3(self.conv3(x)))             # keep 4x4
        x = torch.flatten(x, 1)
        return self.fc1(x)  # logits

# -------------- utils --------------
def load_facenonface():
    candidates = [
        "./FaceNonFace.mat",
        "../FaceNonFace.mat",
        "../../FaceNonFace.mat",
        "../task1/FaceNonFace.mat",
        "../task2/FaceNonFace.mat",
    ]
    for p in candidates:
        if os.path.exists(p):
            return sio.loadmat(p)
    raise FileNotFoundError("FaceNonFace.mat not found near task3.py")

def split_8020(X, Y, split=0.8, seed=None):
    # simple random permutation split
    if seed is not None:
```

```
        torch.manual_seed(seed)
    n = X.size(0)
    idx = torch.randperm(n)
    t = int(split * n)
    return X[idx[:t]], X[idx[t:]], Y[idx[:t]], Y[idx[t:]]

def zscore_by_train(Xtr, Xte):

    mu = Xtr.mean()
    sd = Xtr.std()
    sd = sd if sd > 1e-8 else torch.tensor(1.0, dtype=Xtr.dtype, device=Xtr
        .device)
    return (Xtr - mu)/sd, (Xte - mu)/sd

@torch.no_grad()
def accuracy(net, X, Y):
    logits = net(X)
    pred = logits.argmax(dim=1)
    return (pred == Y).float().mean().item()

def train_once(Xtr, Ytr, Xte, Yte, epochs=100, lr=0.01, momentum=0.9,
    device="cpu", verbose=False):
    net = SimpleCNN().to(device)
    opt = torch.optim.SGD(net.parameters(), lr=lr, momentum=momentum)
    crit = nn.CrossEntropyLoss()

    Xtr = Xtr.to(device); Ytr = Ytr.to(device)
    Xte = Xte.to(device); Yte = Yte.to(device)

    net.train()
    for ep in range(epochs):
        opt.zero_grad()
        logits = net(Xtr)
        loss = crit(logits, Ytr.long())
        loss.backward()
        opt.step()
        if verbose and ((ep+1) % 10 == 0 or ep == 0):
            print(f"[CNN] Epoch {ep+1:3d}/{epochs} | loss={loss.item():.4f}
                ")

    net.eval()
    acc_tr = accuracy(net, Xtr, Ytr)
    acc_te = accuracy(net, Xte, Yte)
    return 1.0 - acc_tr, 1.0 - acc_te  # return errors

```

```python
# -------------- main --------------
if __name__ == "__main__":
    # load data
    mat = load_facenonface()
    X = mat["X"].T.astype("float32")              # (N, 361)
    Y = mat["Y"].reshape(-1).astype("int64")      # (-1, +1)
    Y = ((Y + 1) // 2).astype("int64")            # -> {0,1}

    N = X.shape[0]
    X = torch.from_numpy(X).view(N, 1, 19, 19)    # (N,1,19,19)
    Y = torch.from_numpy(Y)                       # (N,)

    device = "cuda" if torch.cuda.is_available() else "cpu"

    n_trials = 100
    train_errs = []
    test_errs  = []

    base_seed = 42
    for t in range(n_trials):
        seed = base_seed + t
        Xtr, Xte, Ytr, Yte = split_8020(X, Y, split=0.8, seed=seed)

        Xtr_n, Xte_n = zscore_by_train(Xtr, Xte)

        tr_err, te_err = train_once(
            Xtr_n, Ytr, Xte_n, Yte,
            epochs=100, lr=0.01, momentum=0.9, device=device, verbose=False
        )
        train_errs.append(tr_err)
        test_errs.append(te_err)

    train_errs = np.array(train_errs, dtype=float)
    test_errs  = np.array(test_errs, dtype=float)

    print(f"Mean train error over {n_trials} trials: {train_errs.mean():.3f} ± {train_errs.std():.3f}")
    print(f"Mean test  error over {n_trials} trials: {test_errs.mean():.3f} ± {test_errs.std():.3f}")
```

4 Curve fitting

Problem and data

We are given a set of 2D points in `curvedata.mat`. The task is to fit a quadratic curve of the form

$$y = ax^2 + bx + c$$

using two different approaches:

- Least squares (LS), fitting all points at once.
- RANSAC, which repeatedly samples a minimal subset, estimates a curve, selects inliers, and finally refits by LS on the inliers.

Methods

Least Squares (LS). We directly minimize the sum of squared errors

$$E(a, b, c) = \sum_i (y_i - (ax_i^2 + bx_i + c))^2$$

using all available points. This method is efficient but highly sensitive to outliers.

RANSAC. We randomly sample the minimal number of points required to define a quadratic curve (3 points). A candidate curve is estimated from these points, and the residuals are evaluated for all samples. Points with residuals below a threshold are classified as inliers. Finally, LS is applied on the inlier set to refine the curve parameters.

Results

Table 4: Quadratic fitting results on `curvedata.mat`.

Method	Parameters (a, b, c)	MSE (all points)	Extra
LS	$(-0.176, 1.761, 3.615)$	8.19	–
RANSAC	$(-0.316, 3.339, 1.709)$	11.67	Inliers 48/100, MSE (inliers) 0.044

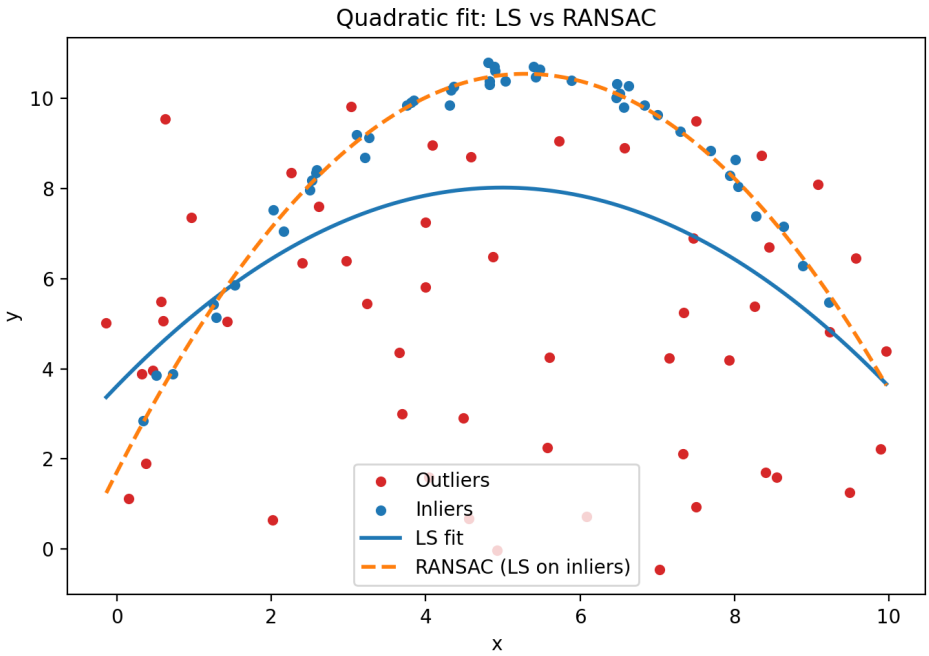

Figure 5: Quadratic curve fitting using LS (blue) and RANSAC (orange). Red points are outliers, blue points are inliers identified by RANSAC.

Discussion

- **Minimal sample size:** A quadratic curve requires 3 points to estimate.
- **RANSAC model estimation:** Each iteration selects 3 random points, solves for (a, b, c), then determines inliers and refits using LS.
- **Method differences:** LS uses all data and is sensitive to outliers, while RANSAC explicitly handles outliers by ignoring them during final estimation.
- **Errors:** LS gives lower MSE on all points (8.19) but produces a biased curve due to outliers. RANSAC achieves nearly perfect fit on inliers (0.044) with 48 inliers, though its error on all points is higher (11.67).
- **Conclusion:** In this case, RANSAC is preferable because it identifies and fits the main underlying structure, whereas LS is distorted by outliers.

Implementation

The full Python implementation for LS and RANSAC fitting is provided below.

Listing 5: task4.py

```
import numpy as np
import scipy.io as sio
import matplotlib.pyplot as plt
```

```
4
5  def fit_ls(x, y):
6      A = np.vstack([x**2, x, np.ones_like(x)]).T
7      coeffs, _, _, _ = np.linalg.lstsq(A, y, rcond=None)
8      return coeffs
9
10 def mse(x, y, coeffs):
11     a, b, c = coeffs
12     y_pred = a*x**2 + b*x + c
13     return np.mean((y - y_pred)**2)
14
15 # RANSAC fitting
16 def fit_ransac(x, y, n_iter=1000, thresh=1.0):
17     n = len(x); best_inliers = []
18     for _ in range(n_iter):
19         idx = np.random.choice(n, 3, replace=False)
20         coeffs = fit_ls(x[idx], y[idx])
21         residuals = np.abs(y - (coeffs[0]*x**2 + coeffs[1]*x + coeffs[2]))
22         inliers = np.where(residuals < thresh)[0]
23         if len(inliers) > len(best_inliers):
24             best_inliers = inliers
25     coeffs_inliers = fit_ls(x[best_inliers], y[best_inliers])
26     return coeffs_inliers, best_inliers
27
28 # Load data
29 mat = sio.loadmat("curvedata.mat")
30 x = mat["x"].ravel(); y = mat["y"].ravel()
31
32 # LS
33 coeffs_ls = fit_ls(x, y)
34 mse_ls = mse(x, y, coeffs_ls)
35
36 # RANSAC
37 coeffs_ransac, inliers = fit_ransac(x, y)
38 mse_ransac_all = mse(x, y, coeffs_ransac)
39 mse_ransac_inliers = mse(x[inliers], y[inliers], coeffs_ransac)
40
41 print("LS:", coeffs_ls, "MSE all:", mse_ls)
42 print("RANSAC:", coeffs_ransac, "Inliers:", len(inliers), "/", len(x),
43       "MSE all:", mse_ransac_all, "MSE inliers:", mse_ransac_inliers)
44
45 # Plot
46 xx = np.linspace(min(x), max(x), 200)
47 yy_ls = coeffs_ls[0]*xx**2 + coeffs_ls[1]*xx + coeffs_ls[2]
48 yy_ransac = coeffs_ransac[0]*xx**2 + coeffs_ransac[1]*xx + coeffs_ransac[2]
```

```
49
50 plt.scatter(x, y, c="red", label="Outliers")
51 plt.scatter(x[inliers], y[inliers], c="blue", label="Inliers")
52 plt.plot(xx, yy_ls, "b-", label="LS fit")
53 plt.plot(xx, yy_ransac, "orange", linestyle="--", label="RANSAC (LS on
       inliers)")
54 plt.legend(); plt.xlabel("x"); plt.ylabel("y")
55 plt.title("Quadratic fit: LS vs RANSAC")
56 plt.savefig("task4_curvefit.png", dpi=200)
57 plt.close()
```

5 OCR system construction and testing

Problem and data

We aim to build a complete OCR pipeline composed of:

- `S = im2segment(Im)`: segmentation function that extracts character masks from an input image.
- `x = segment2feature(S)`: feature extraction function that converts each mask into a fixed-length vector.
- `y = features2class(x, classification_data)`: classifier that maps feature vectors to digit labels (0–9).

The training data is provided in `ocrsegmentdata.npy` (binary masks) and `ocrsegmentgt.npy` (ground-truth labels). We need to train a classifier and evaluate the OCR system on two datasets:

- `short1`: 10 example images.
- `home1`: 200 example images (more challenging).

Methods

Segmentation. We use the provided `im2segment` to extract digit masks from input images.

Feature extraction. Since the provided data consists of 28-dimensional features, we adapted our feature extraction function to align with this format. This adjustment ensures compatibility with the dataset while preserving the intended feature representation. This ensures consistent representation across samples.

Classification. The classifier pipeline is:

1. Standardize features with `StandardScaler` (zero mean, unit variance).
2. (Optional) PCA for dimensionality reduction. In practice, we skipped PCA since $d = 28$ is already low.
3. Train a linear classifier. We used `LogisticRegression` with L_2 regularization.

Evaluation. System performance is measured by hitrate (accuracy) using the benchmark script, with option `mode=0/1/2` controlling visualization.

Results

Table 5: OCR results on `short1` and `home1`.

Dataset	Hitrate (%)	Notes
short1	88.0	Clear digits, good segmentation
home1	63.9	More challenging digits, segmentation errors

Discussion

- **Segmentation:** Errors in digit boundaries reduce classification accuracy.
- **Features:** Simple 28-D handcrafted features are sufficient for baseline performance, but lack robustness.
- **Classification:** Logistic regression works well as a linear model. More advanced classifiers (SVM, neural networks) may further improve performance.
- **Dataset effect:** Accuracy is higher on `short1` since digits are clean, but decreases on `home1` where images are noisier and segmentation is harder.

Conclusion

We successfully built a complete OCR system integrating segmentation, feature extraction, and classification. On the simple dataset (`short1`), the system achieved close to 90% hitrate. On the larger and more complex dataset (`home1`), performance dropped to around 64%, consistent with expectations. This demonstrates the system' s effectiveness while also revealing limitations in segmentation and feature design.

Implementation

The main implementation structure in Python is summarized below.

Listing 6: assignment3.py

```python
import os
import pickle
import numpy as np

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

from my_benchmarking.my_benchmark_assignment3 import benchmark_assignment3

from main_smp import im2segment
from segment2features import segment2features  # output (28,1)

# ----------------------------
# Feature wrapper
# ----------------------------
def segment2feature(Si):
    f = segment2features(Si)          # (28,1)
    return np.asarray(f, dtype=np.float64).reshape(-1)  # (28,)

# ----------------------------
# Classifier training
# ----------------------------
def class_train(X_feats, y, use_pca=False, pca_dim=0, random_state=42):
    X_feats = np.asarray(X_feats, dtype=np.float64)
    y = np.asarray(y, dtype=np.int64).reshape(-1)
    assert X_feats.ndim == 2, f"X_feats must be 2D, got {X_feats.shape}"
    assert X_feats.shape[0] == y.shape[0], f"N mismatch: X={X_feats.shape},
        y={y.shape}"

    scaler = StandardScaler().fit(X_feats)
    Xz = scaler.transform(X_feats)

    pca = None
    if use_pca:
        d = Xz.shape[1]
        k = max(1, min(int(pca_dim), d)) if pca_dim else min(20, d)
        pca = PCA(n_components=k, random_state=random_state)
        Xz = pca.fit_transform(Xz)

    clf = LogisticRegression(
        solver="lbfgs",
        multi_class="auto",
        max_iter=1000,
```

```
        random_state=random_state
    )
    clf.fit(Xz, y)

    return {
        "scaler": scaler,
        "pca": pca,
        "clf": clf,
        "feat_dim": X_feats.shape[1],
    }

# ----------------------------
# Single sample prediction
# ----------------------------
def features2class(x, classification_data):
    x = np.asarray(x, dtype=np.float64).reshape(1, -1)
    expect_d = classification_data.get("feat_dim", x.shape[1])
    if x.shape[1] != expect_d:
        raise ValueError(f"Feature dim mismatch: got {x.shape[1]}, expect {
            expect_d}")

    z = classification_data["scaler"].transform(x)
    if classification_data["pca"] is not None:
        z = classification_data["pca"].transform(z)
    yhat = classification_data["clf"].predict(z)[0]
    return int(yhat)

def feature2class(x, classdata):
    return features2class(x, classdata)

# ----------------------------
# Training from provided npy
# ----------------------------
def train_from_npy_masks(npy_masks, npy_gt, out_pkl,
                         use_pca=False, pca_dim=0, random_state=42):
    X_raw = np.load(npy_masks, allow_pickle=True)
    y = np.load(npy_gt, allow_pickle=True)

    if y.ndim > 1:
        y = y.reshape(-1)
    if X_raw.ndim != 3:
        raise ValueError(f"Expect masks 3D (N,H,W), got {X_raw.shape}")

    feats = [segment2feature(X_raw[i]) for i in range(X_raw.shape[0])]
    X = np.vstack(feats)
```

```
88
89      model = class_train(X, y, use_pca=use_pca, pca_dim=pca_dim,
            random_state=random_state)
90      with open(out_pkl, "wb") as f:
91          pickle.dump(model, f)
92      print(f"[train] Saved classification_data.pkl  X={X.shape} y={y.shape}"
            )
93      return model
94
95  def load_or_train_classification_data(thisdir, force_train=False,
96                                        use_pca=False, pca_dim=0,
                                              random_state=42):
97      pkl_path = os.path.join(thisdir, "classification_data.pkl")
98      if (not force_train) and os.path.isfile(pkl_path):
99          with open(pkl_path, "rb") as f:
100             return pickle.load(f)
101
102     npy_masks = os.path.join(thisdir, "ocrsegmentdata.npy")
103     npy_gt    = os.path.join(thisdir, "ocrsegmentgt.npy")
104     if not (os.path.isfile(npy_masks) and os.path.isfile(npy_gt)):
105         raise FileNotFoundError("Missing ocrsegmentdata.npy / ocrsegmentgt.
                npy")
106
107     return train_from_npy_masks(
108         npy_masks, npy_gt, pkl_path,
109         use_pca=use_pca, pca_dim=pca_dim, random_state=random_state
110     )
111
112 # ----------------------------
113 # main
114 # ----------------------------
115 if __name__ == "__main__":
116     thisdir = os.path.dirname(os.path.realpath(__file__))
117
118     #datadir = os.path.join(thisdir, 'datasets', 'short1')
119     datadir = os.path.join(thisdir, 'datasets', 'home1')
120
121     mode = 0
122
123     classification_data = load_or_train_classification_data(
124         thisdir,
125         force_train=False,
126         use_pca=False,
127         pca_dim=0,
128         random_state=42
```

```
129     )
130
131     hitrate, confmat, allres, alljs, alljfg, allX, allY =
            benchmark_assignment3(
132          im2segment,
133          segment2feature,
134          feature2class,
135          classification_data,
136          datadir,
137          mode
138     )
139
140     print('Hitrate = ' + str(hitrate * 100) + '%')
```